

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab# 02)

Write Lab Title Here

Student Name: Abdulmoeiz Roll No: 24i-6510 Section: CE A

Date performed: 24 August, 2026

MARKS AWARDED: / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task I designed a Full adder circuit. For this task I built a Half Adder module first and then made another full adder module where 2 Half Adders were made to implement sum and carry operation. Later on, to simulate the circuit a test bench module was made with different combinations of inputs. This task was implemented through both gatelevel and dataflow level.

Code:

```
module ha(sum, carry, a, b);
input a, b;
output sum, carry;

xor(sum, a, b);
and(carry, a, b);

endmodule

module FA(SUM, Cout, A, B, Cin);
input A, B, Cin;
output SUM, Cout;

wire Sum1, C1, C2;

ha ha1(Sum1, C1, A, B);
ha ha2(SUM, C2, Sum1, Cin);
or(Cout, C1, C2);

endmodule

module FA_testbench;
reg A,B,Cin;
wire SUM, Cout;

FA FA1(SUM,Cout,A,B,Cin);
initial
begin
A = 0; B = 0; Cin = 0;
#10
A = 0; B = 0; Cin = 1;
#10
```

```
A = 0; B = 1; Cin = 0;
#10
A = 0; B = 1; Cin = 1;
#10
A = 1; B = 0; Cin = 0;
#10
A = 1; B = 0; Cin = 1;
#10
A = 1; B = 1; Cin = 0;
#10
A = 1; B = 1; Cin = 1;
end
endmodule
```

DATAFLOW LEVEL MODELLING:

```
module ha(sum, carry, a, b);
input a, b;
output sum, carry;

assign sum = a^b;
assign carry = a&b;

endmodule

module FA(SUM, Cout, A, B, Cin);
input A, B, Cin;
output SUM, Cout;

wire Sum1, C1, C2;

ha ha1(Sum1, C1, A, B);
ha ha2(SUM, C2, Sum1, Cin);
or(Cout, C1, C2);

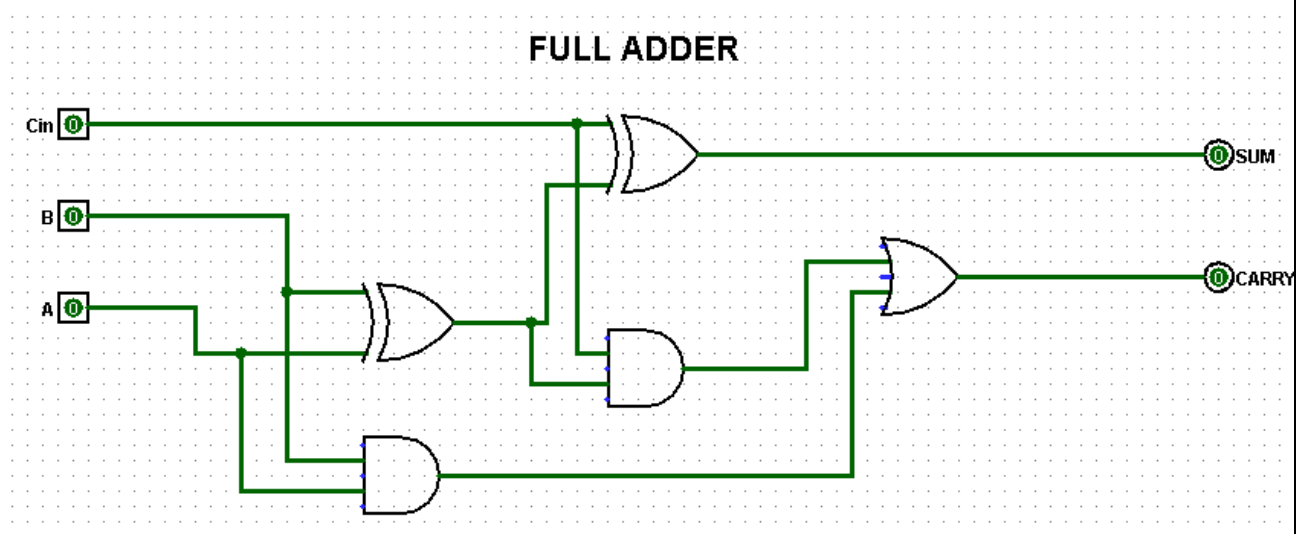
endmodule

module FA_testbench;
reg A,B,Cin;
wire SUM, Cout;

FA FA1(SUM,Cout,A,B,Cin);
initial
begin
A = 0; B = 0; Cin = 0;
#10
A = 0; B = 0; Cin = 1;
```

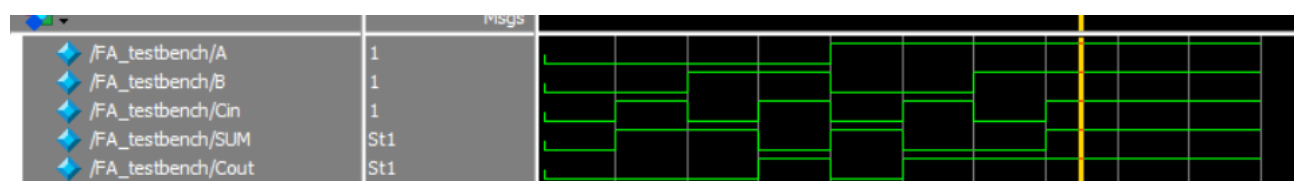
```
#10
A = 0; B = 1; Cin = 0;
#10
A = 0; B = 1; Cin = 1;
#10
A = 1; B = 0; Cin = 0;
#10
A = 1; B = 0; Cin = 1;
#10
A = 1; B = 1; Cin = 0;
#10
A = 1; B = 1; Cin = 1;
end
endmodule
```

LOGISIM CIRCUIT:



Output:

TESTBENCH WAVEFORMS:



LOGISIM Circuit TruthTable :

Inputs	Outputs	Table	Expression	Minimized
Cin	B	A	SUM	CARRY
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Task 2:

Description:

This task was about designed a 3x8 decoder. This task was required to use multiple input and output variables. Also, we included the “enable” bit as a function as well. The code was implemented in gatelevel and dataflow level and a testbench module was written to test our 3x8 decoder and generate it’s waveforms. The circuit was also deisgned on Logisim to ensure correct working via the TruthTable analysis.

Code:

GATELEVEL MODELLING:

```
module decoder( a, b,c,en,o0,o1,o2,o3,o4,o5,o6,o7);
input en,a,b,c;
output o0,o1,o2,o3,o4,o5,o6,o7;
wire na,nb, nc;

not(na,a);
```

```
not(nb,b);
not(nc,c);

and(o0, en, na, nb, nc);
and(o1, en, na, nb, c);
and(o2, en, na, b, nc);
and(o3, en, na, b, c);
and(o4, en, a, nb, nc);
and(o5, en, a, nb, c);
and(o6, en, a, b, nc);
and(o7, en, a, b, c);

endmodule

module Decode_TB;
reg en,a,b,c;
wire o0,o1,o2,o3,o4,o5,o6,o7;

decoder d1(a,b,c,en,o0,o1,o2,o3,o4,o5,o6,o7);

initial
begin
en = 0; a = 0; b = 0; c = 0;
#10
en = 1; a = 0; b = 0; c = 0;
#10
en = 1; a = 0; b = 0; c = 1;
#10
en = 1; a = 0; b = 1; c = 0;
#10
en = 1; a = 0; b = 1; c = 1;
#10
en = 1; a = 1; b = 0; c = 0;
#10
en = 1; a = 1; b = 0; c = 1;
#10
en = 1; a = 1; b = 1; c = 0;
#10
en = 1; a = 1; b = 1; c = 1;
end
endmodule
```

DATAFLOW LEVEL MODELLING:

```
module decoder( a, b,c,en,o0,o1,o2,o3,o4,o5,o6,o7);
input en,a,b,c;
output o0,o1,o2,o3,o4,o5,o6,o7;
wire na,nb, nc;
```

```
assign na =~a;
assign nb =~b;
assign nc =~c;

not(nc,c);

assign o0 = na& nb& nc &en;
assign o1 = na& nb& c &en;
assign o2 = na& b& nc &en;
assign o3 = na& b& c &en;
assign o4 = a& nb& nc &en;
assign o5 = a& nb& c &en;
assign o6 = a& b& nc &en;
assign o7 = a& b& c &en;

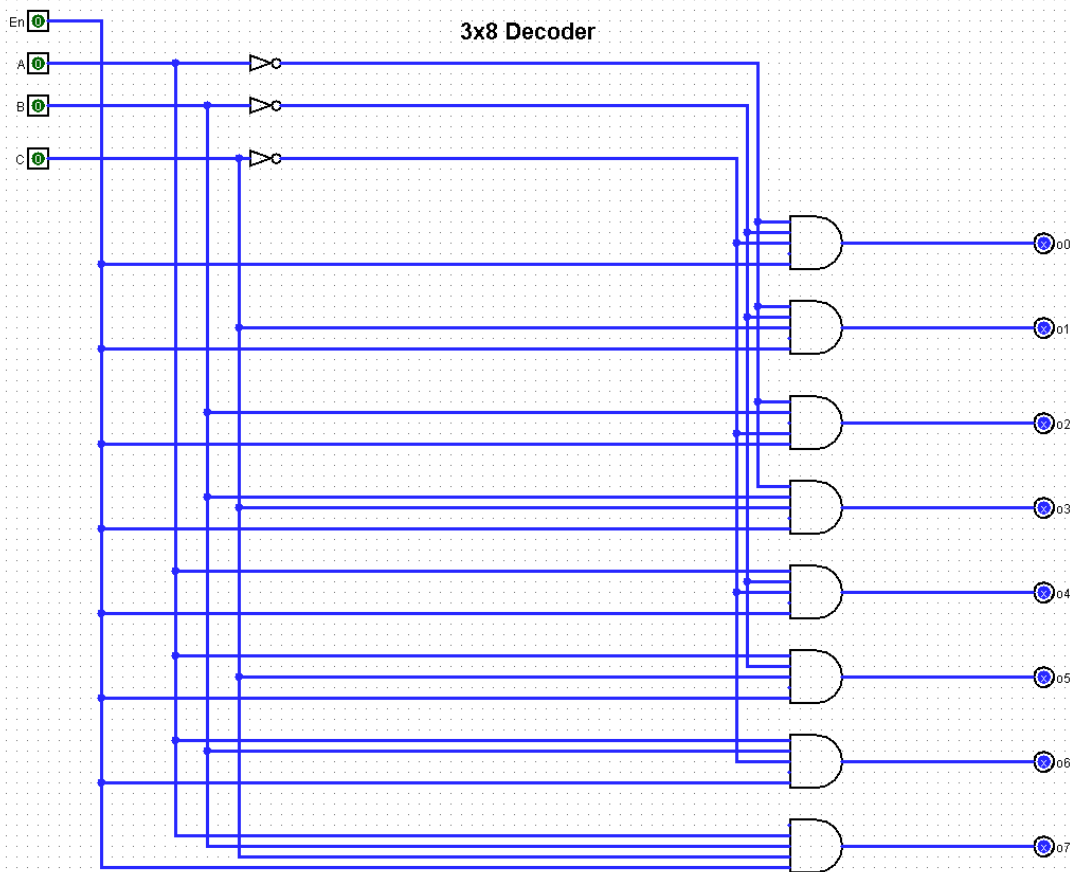
endmodule

module Decode_TB;
reg en,a,b,c;
wire o0,o1,o2,o3,o4,o5,o6,o7;

decoder d1(a,b,c,en,o0,o1,o2,o3,o4,o5,o6,o7);

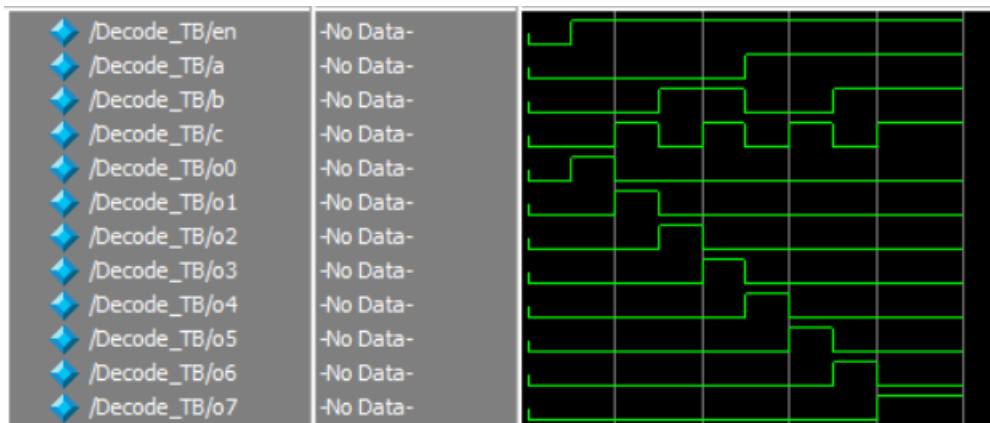
initial
begin
en = 0; a = 0; b = 0; c = 0;
#10
en = 1; a = 0; b = 0; c = 0;
#10
en = 1; a = 0; b = 0; c = 1;
#10
en = 1; a = 0; b = 1; c = 0;
#10
en = 1; a = 0; b = 1; c = 1;
#10
en = 1; a = 1; b = 0; c = 0;
#10
en = 1; a = 1; b = 0; c = 1;
#10
en = 1; a = 1; b = 1; c = 0;
#10
en = 1; a = 1; b = 1; c = 1;
end
endmodule
```

LOGISIM CIRCUIT DESIGN:



Output:

TESTBENCH WAVEFORM:



LOGISIM CIRCUIT TRUTHTABLE:

En	A	B	C	o0	o1	o2	o3	o4	o5	o6	o7
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0	0	0	0
1	0	0	1	0	1	0	0	0	0	0	0
1	0	1	0	0	0	1	0	0	0	0	0
1	0	1	1	0	0	0	1	0	0	0	0
1	1	0	0	0	0	0	0	1	0	0	0
1	1	0	1	0	0	0	0	0	1	0	0
1	1	1	0	0	0	0	0	0	0	1	0
1	1	1	1	0	0	0	0	0	0	0	1

Task 3:

Description:

// Write description of the task here  (max. five to seven lines)

This task implements an 8x1 multiplexer using gate-level modeling in Verilog. Basic NOT, AND, and OR gates are used to build the multiplexer circuit. A testbench is also written to apply different select inputs and check the output. The simulation helps verify that the correct input is selected for each combination of select lines.

Code:

```
GATE LEVEL
module mux8to1_gate(
    I0, I1, I2, I3, I4, I5, I6, I7,
    SEL2, SEL1, SEL0, OUT
);

input I0, I1, I2, I3, I4, I5, I6, I7;
input SEL2, SEL1, SEL0;
output OUT;

wire NSEL2, NSEL1, NSEL0;
wire W0, W1, W2, W3, W4, W5, W6, W7;
```

```
not (NSEL2, SEL2);
not (NSEL1, SEL1);
not (NSEL0, SEL0);
and (W0, NSEL2, NSEL1, NSEL0, I0);
and (W1, NSEL2, NSEL1, SEL0, I1);
and (W2, NSEL2, SEL1, NSEL0, I2);
and (W3, NSEL2, SEL1, SEL0, I3);
and (W4, SEL2, NSEL1, NSEL0, I4);
and (W5, SEL2, NSEL1, SEL0, I5);
and (W6, SEL2, SEL1, NSEL0, I6);
and (W7, SEL2, SEL1, SEL0, I7);
or (OUT, W0, W1, W2, W3, W4, W5, W6, W7);
```

```
endmodule
```

```
module mux8to1_gate_tb;
```

```
reg IN0, IN1, IN2, IN3, IN4, IN5, IN6, IN7;
reg SEL0_TB, SEL1_TB, SEL2_TB;
wire OUT_TB;
```

```
mux8to1_gate uut(
    IN0, IN1, IN2, IN3, IN4, IN5, IN6, IN7,
    SEL2_TB, SEL1_TB, SEL0_TB, OUT_TB
);
```

```
initial
```

```
begin
```

```
    IN0=0; IN1=1; IN2=0; IN3=1;
    IN4=0; IN5=1; IN6=0; IN7=1;
```

```
    SEL2_TB=0; SEL1_TB=0; SEL0_TB=0;
    #50;
```

```
    SEL2_TB=0; SEL1_TB=0; SEL0_TB=1;
    #50;
```

```
    SEL2_TB=0; SEL1_TB=1; SEL0_TB=0;
    #50;
```

```
    SEL2_TB=0; SEL1_TB=1; SEL0_TB=1;
    #50;
```

```
    SEL2_TB=1; SEL1_TB=0; SEL0_TB=0;
    #50;
```

```
    SEL2_TB=1; SEL1_TB=0; SEL0_TB=1;
```

```
#50;

SEL2_TB=1; SEL1_TB=1; SEL0_TB=0;
#50;

SEL2_TB=1; SEL1_TB=1; SEL0_TB=1;
#50;
end

endmodule

DATA FLOW LEVEL
module mux8to1(
    I0, I1, I2, I3, I4, I5, I6, I7,
    SEL2, SEL1, SEL0, OUT
);

input I0, I1, I2, I3, I4, I5, I6, I7;
input SEL2, SEL1, SEL0;
output OUT;

wire NSEL2, NSEL1, NSEL0;
wire W0, W1, W2, W3, W4, W5, W6, W7;

assign NSEL2 = ~SEL2;
assign NSEL1 = ~SEL1;
assign NSEL0 = ~SEL0;

assign W0 = NSEL2 & NSEL1 & NSEL0 & I0;
assign W1 = NSEL2 & NSEL1 & SEL0 & I1;
assign W2 = NSEL2 & SEL1 & NSEL0 & I2;
assign W3 = NSEL2 & SEL1 & SEL0 & I3;
assign W4 = SEL2 & NSEL1 & NSEL0 & I4;
assign W5 = SEL2 & NSEL1 & SEL0 & I5;
assign W6 = SEL2 & SEL1 & NSEL0 & I6;
assign W7 = SEL2 & SEL1 & SEL0 & I7;

assign OUT = W0 | W1 | W2 | W3 | W4 | W5 | W6 | W7;

endmodule

module mux8to1_tb;

reg IN0, IN1, IN2, IN3, IN4, IN5, IN6, IN7;
reg SEL0_TB, SEL1_TB, SEL2_TB;
wire OUT_TB;

mux8to1 uut(
```

```
    IN0, IN1, IN2, IN3, IN4, IN5, IN6, IN7,  
    SEL2_TB, SEL1_TB, SEL0_TB, OUT_TB  
);
```

```
initial
```

```
begin
```

```
    IN0=0; IN1=1; IN2=0; IN3=1;
```

```
    IN4=0; IN5=1; IN6=0; IN7=1;
```

```
    SEL2_TB=0; SEL1_TB=0; SEL0_TB=0;
```

```
    #50;
```

```
    SEL2_TB=0; SEL1_TB=0; SEL0_TB=1;
```

```
    #50;
```

```
    SEL2_TB=0; SEL1_TB=1; SEL0_TB=0;
```

```
    #50;
```

```
    SEL2_TB=0; SEL1_TB=1; SEL0_TB=1;
```

```
    #50;
```

```
    SEL2_TB=1; SEL1_TB=0; SEL0_TB=0;
```

```
    #50;
```

```
    SEL2_TB=1; SEL1_TB=0; SEL0_TB=1;
```

```
    #50;
```

```
    SEL2_TB=1; SEL1_TB=1; SEL0_TB=0;
```

```
    #50;
```

```
    SEL2_TB=1; SEL1_TB=1; SEL0_TB=1;
```

```
    #50;
```

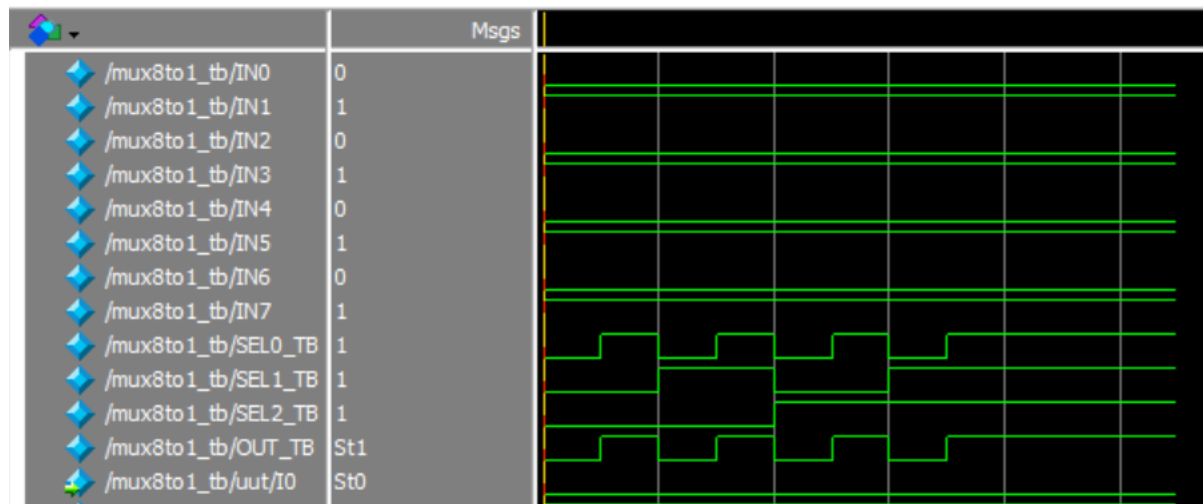
```
end
```

```
endmodule
```

Output:

```
// Import output here 
```

DATA FLOW LEVEL



GATE LEVEL

